

A deep reinforcement learning assisted adaptive genetic algorithm for flexible job shop scheduling

Jian Ma^a, Weinan Gao^{a,*,*}, Weitian Tong^b

^a State Key Laboratory of Synthetical Automation for Process Industries, Northeastern University, Shenyang, 110819, China

^b Department of Computer Science, Georgia Southern University, Statesboro, GA 30458, USA

ARTICLE INFO

Keywords:

Deep reinforcement learning
Flexible job shop scheduling problem
Genetic algorithm
Mutation operations
Parameter configuration

ABSTRACT

The flexible job shop scheduling problem (FJSP) is a challenging combinatorial optimization problem in manufacturing systems. Existing intelligent optimization algorithms for FJSP are often hard to tune key parameters and operations efficiently, which losses the optimality of the obtained solution. To address the issues of genetic algorithms (GA) being prone to local optima and slow convergence, this paper proposes a deep reinforcement learning-assisted adaptive genetic algorithm (DRL-A-GA) for solving FJSP. In the proposed algorithm, continuous state vectors are used to represent the population state of the GA, and four mutation operations with respect to FJSP are designed as actions. Deep reinforcement learning is employed to adaptively tune the key parameters of the GA and dynamically select appropriate genetic operations. To validate the performance of DRL-A-GA, three sets of benchmark instances are selected for testing, and the results are compared with those of classical optimization algorithms and hybrid algorithms. The experimental results demonstrate that the proposed DRL-A-GA significantly outperforms both traditional optimization and intelligent hybrid optimization algorithms for solving FJSP, effectively improving solution quality and accelerating convergence.

1. Introduction

Manufacturing systems play a critical role in industrial production, and scheduling problems become one of the most prevalent hurdles in manufacturing industry. Among the various types of scheduling problems, the job shop scheduling problem (JSP) is one of the most challenging combinatorial optimization and NP-hard problems (Garey et al., 1976).

The JSP concerns creating an optimal scheduling plan for a set of jobs that must be processed on specific machines in a defined order to achieve certain performance objectives, such as minimizing total completion time, balancing machine workloads, and reducing energy consumption (Tang et al., 2011). The flexible job shop scheduling problem (FJSP) is an advancement of the traditional JSP, offering enhanced flexibility in scheduling by allowing operations to be assigned to multiple machines, with each machine capable of performing different operations (Brandimarte, 1993). Therefore, FJSP requires not only the sequencing of operations but also the decision-making for machine selection, making it more complex than the traditional JSP. Moreover, FJSP is more aligned with the actual needs of modern manufacturing systems (Fattahi et al., 2007), such as aerospace parts manufacturing,

semiconductor production, and chemical processing (Sundar et al., 2017).

The methods for solving the FJSP can be categorized into exact and approximate approaches. Berrada and Stecké (1986) proposed an exact method branch and bound (B&B) algorithm for balancing workloads in flexible manufacturing systems. Soto et al. (2020) introduced an innovative parallel B&B algorithm to solve multi-objective FJSP. Mixed integer linear programming (MILP) models are also an effective method for solving FJSP. Gomes et al. (2005) presented a novel MILP model for scheduling make-to-order flexible job shops, incorporating product recirculation into the model. Recently, constraint programming (CP) has become increasingly recognized as an exact approach for solving FJSP. (Meng et al., 2020) proposed four MILP models and one CP model to address the distributed FJSP (DFJSP).

It is important to note that the aforementioned exact methods can efficiently solve small-scale FJSP problems within a reasonable amount of time. However, as the problem size increases, both spatial and temporal complexities the exact methods grow rapidly, rendering them too complex to solve real-world problems. In contrast, approximate methods can find near-optimal solutions within a reasonable time, which is more practical.

* Corresponding author.

E-mail addresses: 2300929@stu.neu.edu.cn (J. Ma), gaown@mail.neu.edu.cn (W. Gao), wtong@georgiasouthern.edu (W. Tong).

<https://doi.org/10.1016/j.engappai.2025.110447>

Received 30 October 2024; Received in revised form 23 January 2025; Accepted 25 February 2025

Available online 13 March 2025

0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Traditional approximate methods for FJSP involve basic dispatching rules (DRs) ((Jeong and Kim, 1998), (O'Keefe and Kasirajan, 1992), (Shanker and Tzen, 1985)), and some optimization algorithms, such as tabu search (TS), simulated annealing (SA), genetic algorithm (GA), particle swarm optimization (PSO), ant colony optimization (ACO) ((Brandimarte, 1993), (Palmer, 1996), (Kacem et al., 2002), (Ding and Gu, 2020b), (Sutton and Barto, 1998), (Jiang and Zhang, 2018)). Among these approximate methods, GA as an evolutionary algorithm, is a gradient-free black-box optimization technique. Due to its strong search capabilities, many researchers have used GA to solve FJSP. Chen et al. (1999) proposed a GA to solve the FJSP problem. This algorithm introduces a novel chromosome encoding method that combines routing policies and operation sequencing, along with various genetic operators. (Kacem et al., 2002) developed a GA based on the approach of localization. This method generates an assignment model that controls the execution of the GA to solve the FJSP. Gen et al. (2005) proposed a multistage GA for FJSP, incorporating phenotype-based crossover and mutation operations to optimize scheduling by leveraging the problem's specific characteristics. Pezzella et al. (2008) developed an improved GA to solve the FJSP. This algorithm integrates multiple strategies to optimize the generation of the initial population, the selection of individuals, and the process of regenerating new individuals.

GA is highly sensitive to hyperparameters like crossover and mutation rates, where even small changes can significantly impact performance. To address this, some studies have enhanced GA's effectiveness by dynamically selecting operators and adjusting hyperparameters. Srinivas and Patnaik (1994) applied an adaptive GA for multimodal function optimization, where crossover and mutation probabilities vary based on solution fitness values. Liu et al. (2006) proposed a hybrid algorithm combining GA and Taguchi method to solve FJSP, where Taguchi method is used to select better genes during crossover operation. Wang and Tang (2011) proposed an improved adaptive GA for solving JSP, inspired by hormone modulation mechanisms, designing adaptive crossover and mutation rates, and employing a new crossover operation method. Chang and Liu (2017) presented a hybrid GA to solve DFJSP, using the Taguchi method to optimize GA parameters and employing various crossover and mutation operations. Chen et al. (2020) proposed a self-learning GA, using reinforcement learning (RL) to intelligently tune key GA parameters, employing state-action-reward-state-action and Q -learning algorithms as initial and later learning methods, respectively. Song et al. (2023b) presented a deep reinforcement learning (DRL) based GA to solve the electromagnetic detection satellite scheduling problem, embedding Q -learning into an improved GA to guide population search through the selection of evolutionary operations.

Although the performance of GA has been improved in above-mentioned work, most studies only focus on either adaptive changes of key parameters in GA or the dynamic selection of evolutionary operations, without simultaneously adaptive changes to both. In addition, most existing methods use general evolutionary operators and do not design evolutionary operators exactly for FJSP, resulting in the inability of the population to evolve according to the characteristics of the problem and reducing the optimization efficiency of the algorithm. For evolutionary algorithms such as GA, the design and selection of evolutionary operators will significantly affect their performance. Choosing appropriate combinations of evolutionary operators and parameter settings at different stages of GA can enhance their global search capability, enabling them to find optimal solutions more efficiently. Moreover, using continuous state vectors can more comprehensively represent the different evolutionary stages of the genetic algorithm's population. This enables the agent to dynamically tune parameters and evolutionary operations based on the current stage, thereby further improving the overall optimization capability of the algorithm. Table 1 presents a comparison between our method and other reinforcement learning-assisted evolutionary algorithms for scheduling

optimization problems, focusing on their similarities and differences. Our approach utilizes a continuous state space, enabling adaptive adjustment of key genetic algorithm parameters and dynamic selection of evolutionary operators, which significantly improves algorithm performance, capabilities that are not achieved by other methods.

This paper proposes a DRL-assisted-GA (DRL-A-GA) to solve FJSP, aiming to minimize makespan. We use continuous state vectors to represent the current population state of GA, and design four mutation operators specific to FJSP. The DRL assists the GA in selecting appropriate parameters and evolutionary operators at different iterative stages.

Here are the main contributions of our research:

- We design four specialized mutation operations for GA based on the characteristics of the FJSP. These operations include machine selection mutation based on processing time, machine selection mutation based on the maximum machine load, machine selection mutation based on average machine load and machine selection mutation based on critical path. These mutation operations can be selected to pair with various stages of GA, thereby further enhancing its performance in solving the FJSP.
- We propose an adaptive GA based on DRL. To be more specific, DRL to assist the GA in selecting critical parameters (crossover rate and mutation rate) and genetic operators (crossover and mutation operations), thereby identifying higher quality solutions. In the DRL, we constructed a four-dimensional state vector to represent the current state of the GA and proposed a reward function specific to FJSP to guide the GA in finding optimal solutions.
- The effectiveness of the proposed algorithm is validated through a series of experiments, which included three commonly used small-scale Kacem benchmark instances (Kacem et al., 2002), ten Brandimarte benchmark instances (Brandimarte, 1993), and the additional 20 Fattahi (Fattahi et al., 2007) instances.

The remainder of this paper is organized as follows. In Section 2 we construct the FJSP model and introduce the preliminaries. In Section 3, we provide a detailed explanation of the proposed algorithm. Section 4, we validate the performance of the proposed algorithm on benchmark instances, along with a discussion and analysis. Finally, Section 5 concludes the paper and suggests future research directions.

Notations. Throughout this paper, $\mathbb{R}_{\geq 0}$ denotes the set of nonnegative real numbers, while $\mathbb{Z}_{\geq 0}$ represents the set of nonnegative integers. The notation $\mathbb{Z}_{\geq 0}^{2T_o}$ refers to a vector of nonnegative integers of dimension $2T_o$. Similarly, $\mathbb{R}_{\geq 0}^{I \times J \times K}$ represents a tensor of dimension $I \times J \times K$, containing nonnegative real values.

2. Problem formulation and preliminaries

In this Section, we first describe the problem and develop a mathematical model, then introduce the principles of GA and DRL, and explore the integration strategy of GA with DRL.

2.1. Problem description

The FJSP is a complex and advanced extension of the classical JSP. Compared to JSP, FJSP requires not only determining the sequence of operations for each job but also selecting the appropriate machine for each operation, thereby providing greater scheduling flexibility. Table 2 illustrates an example of FJSP, which involves three jobs (J_1 , J_2 , and J_3) and five machines available for processing. Table 3 lists the definitions of symbols used in the FJSP model, where J_i represents the i th job, O_{ij} represents the j th operation of the i th job, and M_k represents the k th machine.

In this example, job J_1 consists of two operations, O_{11} and O_{12} ; job J_2 also consists of two operations, O_{21} and O_{22} ; while job J_3 includes

Table 1

Comparison of existing methods for scheduling optimization problems using reinforcement learning-assisted evolutionary algorithms.

Reference	Problem	Algorithm	State space	Evolutionary operations	Critical parameters
Song et al. (2023b)	EDSSP	RL-GA	Discrete	Yes	No
Tian et al. (2023)	MOPS	MOEA/D-DQN	Continuous	Yes	No
Jiang et al. (2023)	FJSP	ISLGA	Discrete	Yes	No
Song et al. (2023a)	EESSP	LAGA	Continuous	No	Yes
Chen et al. (2020)	FJSP	SLGA	Discrete	No	Yes
Our Proposed	FJSP	DRL-GA	Continuous	Yes	Yes

Table 2

FJSP instance.

Job	Operation	Machine				
		M_1	M_2	M_3	M_4	M_5
J_1	O_{11}	2	–	4	5	2
	O_{12}	–	3	2	–	–
J_2	O_{21}	2	4	–	–	4
	O_{22}	6	8	–	5	–
J_3	O_{31}	–	3	6	–	5
	O_{32}	–	4	6	5	–
	O_{33}	7	5	8	–	10

Table 3

Symbols and definitions for FJSP modeling.

Symbol	Definition
n	The number of jobs
m	The number of machines
i	The index of jobs
j	The index of operations
k	The index of machines
p_i	Number of operations of job i
$O_{i,j}$	The j th operation of job i
M_k	Machine k
$M_{i,j}$	Set of machines that can process the j th operation of job i
$X_{i,j}^k$	Decision variable, 1 if $O_{i,j}$ is processed on M_k , otherwise 0
t	Time
$P(O_{i,j})$	Processing time of $O_{i,j}$
$P_{i,j,k}$	Processing time of the j th operation of job i on M_k
$S(O_{i,j})$	Start time of $O_{i,j}$
$F(O_{i,j})$	Completion time of $O_{i,j}$
MT_k	The k th machine's time of processing
c_k	The completion time of machine k
$C_{\max}$	The maximum completion time of all jobs

three operations, O_{31} , O_{32} , and O_{33} . Each operation can be processed on multiple machines, reflecting the core feature of flexible scheduling. The numbers in the table indicate the processing time required for each operation on a specific machine, while the symbol “–” indicates that an operation cannot be processed on that machine. Solving FJSP involves selecting machine assignments and operation sequences to optimize a predefined cost, such as minimizing total completion time or balancing machine workloads.

2.2. Mathematical modeling

In this paper, the goal of FJSP is to minimize the maximum completion time of all jobs ($C_{\max}$), i.e.,

$$\min(C_{\max}) = \min \left(\max_{k=1, \dots, m} c_k \right) \quad (1)$$

The following are some constraints to be constructed:

- Single Machine Processing Constraint:

$$\sum_{i=1}^n \sum_{j=1}^{p_i} X_{i,j}^k(t) \leq 1 \quad X_{i,j}^k \in \{0, 1\}, k \in M_{i,j}, \forall t \in \mathbb{R}^+ \quad (2)$$

which states each machine can process at most one operation at the same time.

- Machine Selection Constraint:

$$\sum_{k \in M_{i,j}} X_{i,j}^k = 1 \quad \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, p_i\}, X_{i,j}^k \in \{0, 1\} \quad (3)$$

which implies that each operation can only be processed on one machine from a specified set of machines.

- Operation Sequence Constraint:

$$S(O_{i,j+1}) \geq F(O_{i,j}) \quad \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, p_i - 1\} \quad (4)$$

which indicates that each job's operations must follow a predetermined order, i.e., the j th operation must be completed before the $(j+1)$ th operation begins.

- Processing Time Constraint:

$$P(O_{i,j}) = P_{i,j,k} \quad \text{if } X_{i,j}^k = 1, \\ \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, p_i\}, \forall k \in \{1, \dots, m\} \quad (5)$$

which implies that the processing time of each operation on a machine is fixed.

- Start Time Constraint:

$$S(O_{i,j}) \geq F(O_{i,j-1}) \quad \forall i \in \{1, \dots, n\}, \forall j \in \{2, \dots, p_i\} \quad (6)$$

which means that the start time of each operation must be later than the completion of the previous operation.

- Completion Time Constraint:

$$F(O_{i,j}) = S(O_{i,j}) + P(O_{i,j}) \quad \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, p_i\} \quad (7)$$

- Total Job Quantity Constraint:

$$\sum_{j=1}^{p_i} \sum_{k \in M_{i,j}} X_{i,j}^k = p_i \quad \forall i \in \{1, \dots, n\} \quad (8)$$

which implies that during scheduling, each job must be fully processed, with no parts omitted or duplicated.

2.3. GA and DRL

We will provide a brief overview of GA and DRL, and explain the integration of DRL and GA.

2.3.1. Genetic algorithm

The GA is an optimization algorithm based on the principles of natural selection and genetics, designed to enhance the quality of a population by simulating the process of biological evolution. GA can be divided into the following steps: individual encoding and decoding, population initialization, fitness evaluation, selection, crossover, mutation, and termination criteria. Fig. 1 shows the flowchart of GA.

2.3.2. Deep reinforcement learning

RL is a machine learning method used to solve sequential decision-making problems. These problems are usually modeled as a markov decision process (MDP), defined as $\langle S, A, P, R, \gamma \rangle$, where: S represents the state space; A is the action space; $P(s'|s, a)$ is the state transition probability function, which represents the probability of transition to state s' after taking action a in state s ; $R(s, a)$ denotes the

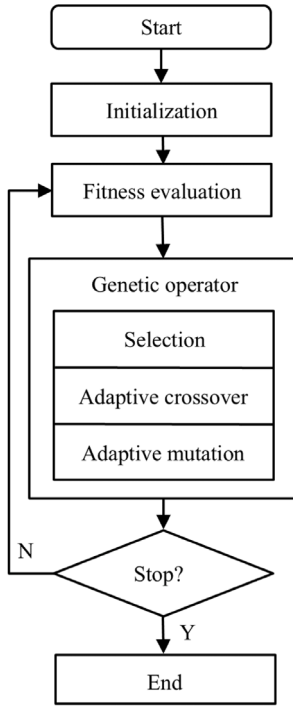


Fig. 1. GA flowchart.

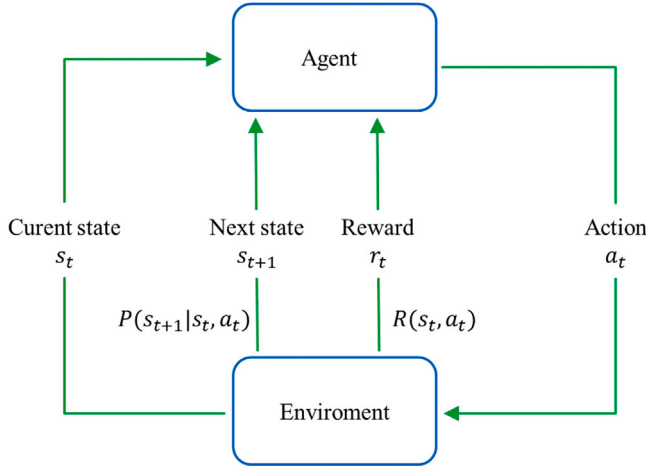


Fig. 2. RL framework.

reward function, which represents the immediate reward obtained by the agent after taking action a in state s ; $\gamma \in [0, 1)$ is the discount factor that weighs the present value of future rewards. This process can be described as an interaction between agent and environment as shown in Fig. 2. Q -learning is a classic RL algorithm that optimizes a parameterized Q -function to estimate the value of state-action pairs. However, traditional Q -learning is challenging to handle large state-action spaces. To address this issue, Mnih et al. (2013) introduced a DRL algorithm, DQN. Compared to traditional RL, DRL processes complex high-dimensional inputs through deep neural networks, enabling it to make decisions and optimize strategies in complex and continuous environments.

2.3.3. GA and DRL integration

Since each GA iteration can be modeled as an MDP, the current population (state) depends only on the previous generation and the genetic

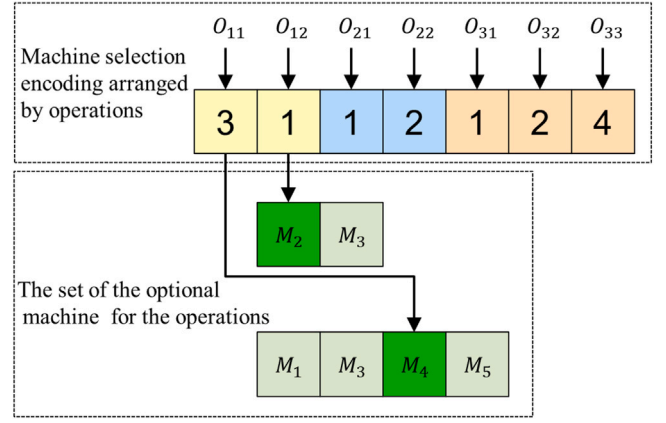


Fig. 3. MS encoding.

operations (selection, crossover, mutation), without relying on earlier generations. This evolutionary process resembles the state transitions in MDPs, where the fitness values can be used to compute rewards, guiding the population towards better solutions. Consequently, this paper incorporates DRL to assist GA in achieving adaptive adjustment of parameters and genetic operations, thereby enhancing the algorithm's adaptability and improving optimization performance.

3. The DRL-Assisted-GA algorithm

In this Section, we propose a novel DRL-Assisted-GA algorithm to solve FJSP. We will describe key components of the algorithm, including chromosome representation, population initialization, fitness evaluation, crossover and mutation operators, and other GA steps. Additionally, we will introduce core DRL elements, such as action selection, reward mechanisms, environment settings, and learning strategies.

3.1. Chromosome representation

When using GA to solve the FJSP, each chromosome represents a scheduling scheme. This paper mainly adopts the encoding method proposed by Zhang et al. (2011) to ensure the effectiveness of the encoding and decoding processes. Each chromosome is divided into two components: the machine selection (MS) and the operation sequence (OS). These two components use different encoding and decoding methods, and the total length of the chromosome is $2T_o$, T_o represents the total number of operations of all jobs.

In the MS , each integer represents the index of the selected machine from its feasible set. In the OS , we adopt an operation-based representation, which determines the processing order of each operation within each job.

For example, in the instance shown in Table 2, a possible encoding for the MS and OS is illustrated in Figs. 3 and 4. The complete scheduling scheme is shown in Fig. 5, the combination of the MS and OS yields the following scheduling scheme:

$$\{(O_{11}, M_4), (O_{12}, M_2), (O_{21}, M_1), (O_{22}, M_2), (O_{31}, M_2), (O_{32}, M_3), (O_{33}, M_5)\}.$$

The corresponding operation sequence is:

$$O_{21} \rightarrow O_{11} \rightarrow O_{31} \rightarrow O_{22} \rightarrow O_{32} \rightarrow O_{33} \rightarrow O_{12}.$$

Remark 1. The chromosome structure in this paper, divided into MS and OS , enhances the flexibility and efficiency of the GA. By independently encoding MS (Fig. 3) and OS (Fig. 4) specifically, this approach supports efficient decoding and complex scheduling solutions, making it well-suited for the FJSP.

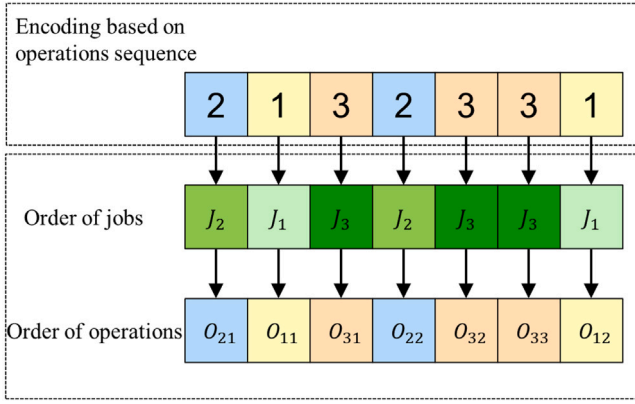


Fig. 4. OS encoding.

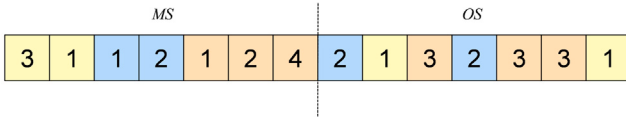


Fig. 5. Individual chromosome representation: MS and OS.

3.2. Population initialization

Population initialization is the first step in the GA. In this paper, we employ three different initialization methods proposed by Zhang et al. (2011) to generate the initial population, namely global initialization, local initialization, and random initialization.

The global initialization method selects the optimal machine by recording the total processing time of each machine throughout the entire processing stage. At each step, a job is randomly selected, starting from its first operation. Each operation is then assigned to the machine with the shortest processing time. In cases where multiple machines have the same processing time, one is randomly selected. After completing an operation, the processing time of the corresponding machine is updated. The process continues by assigning machines to subsequent operations of the job until all operations of all jobs are completed.

The primary difference between the local initialization method and the global initialization method lies in the scope of recorded processing times. The local initialization method only considers the processing stage of the current job, without accounting for the total processing time across all stages. In this method, the process begins with the first operation of a selected job, and the machine with the shortest processing time during the current stage is selected. Once all operations of the current job are completed, the processing times of all machines are reset to zero when transitioning to the next job. This ensures that the processing times for each job are calculated independently.

Random initialization does not rely on specific optimization criteria and selects machines randomly for each operation, ensuring population diversity. These three methods primarily focus on the initialization of the MS, while the OS is randomly generated.

The average fitness $\bar{f}_{init}$, optimal fitness f_{init}^* , and fitness variance f_{init}^{var} of the initial population will be used to represent the state of populations. The mathematical symbols and their explanations involved in the state representation are shown in Table 4.

Remark 2. Utilizing global, local, and random initialization methods enhances the diversity and balance of the initial population. Global and local initialization promote load balancing and job-specific optimization, while random initialization increases population diversity.

Table 4

Summary of symbols used in the state feature vectors.

Symbol	Explanation
N	The total number of individuals in the population
$\bar{f}_{init}$	The mean fitness of the initial population
f_{init}^*	The best fitness of the initial population
f_{init}^{var}	The standard deviation of fitness of the initial population
$\bar{f}_{norm}$	The mean fitness of the current population normalized by $\bar{f}_{init}$
f_{norm}^*	The best fitness of the current population normalized by f_{init}^*
f_{norm}^{var}	The standard diversity of fitness of the current population normalized by f_{init}^{var}
i_{prog}	The progress of the current iteration relative to the total number of iterations
$\bar{f}$	The average fitness of the current population
f^*	The best fitness of the current population
f^{var}	The deviation of fitness within the current population
i_{cur}	The current iteration number within the evolutionary process
i_{max}	The total number of iterations allowed in the evolutionary process
f	The fitness set of the current population
f^i	The fitness value of the i -th individual within the population

3.3. Fitness evaluation

Fitness is a key criterion for evaluating chromosome quality and is essential to the GA. In this paper, our objective is to minimize the makespan. Therefore, in this paper, the fitness function is defined as the makespan:

$$f^i = \min(C_{max}^i) \quad (9)$$

Consequently, a smaller fitness value indicates a better scheduling solution.

In the DRL-A-GA algorithm, we define fitness-based state feature vectors to represent the current population state and normalize them to eliminate dimensional differences. Specifically, the state features include the following aspects:

Average Fitness of the Current Population ($\bar{f}_{norm}$): This metric reflects the overall fitness level of the entire population. The normalization formula is as follows:

$$\bar{f}_{norm} = \frac{\bar{f}}{\bar{f}_{init}} = \frac{\frac{1}{N} \sum_{i=1}^N f^i}{\bar{f}_{init}} \quad (10)$$

Best Fitness of the Current Population (f_{norm}^*): This metric reflects the performance of the best individual in the population. The normalization formula is as follows:

$$f_{norm}^* = \frac{f^*}{f_{init}^*} = \frac{\min(f)}{f_{init}^*} \quad (11)$$

Fitness Standard Deviation of the Current Population (f_{norm}^{var}): This metric is used to measure the diversity of the population. A larger standard deviation indicates a higher diversity in fitness among individuals in the population, while a smaller standard deviation indicates convergence of the population. The normalization formula is as follows:

$$f_{norm}^{var} = \frac{f^{var}}{f_{init}^{var}} = \frac{\sqrt{\frac{1}{N} \sum_{i=1}^N (f^i - \bar{f})^2}}{f_{init}^{var}} \quad (12)$$

Current Iteration Step (i_{prog}): This metric is used to represent the progress of the current iteration. The normalization formula is as follows:

$$i_{prog} = \frac{i_{cur}}{i_{max}} \quad (13)$$

By applying these normalization processes, the final state vector is defined as: $\{f_{norm}^*, \bar{f}_{norm}, f_{norm}^{var}, i_{prog}\}$.

Remark 3. In this paper, the state is defined as a continuous state space $\{f_{norm}^*, \bar{f}_{norm}, f_{norm}^{var}, i_{prog}\}$, which more accurately reflects the characteristics of the current GA population. This representation allows

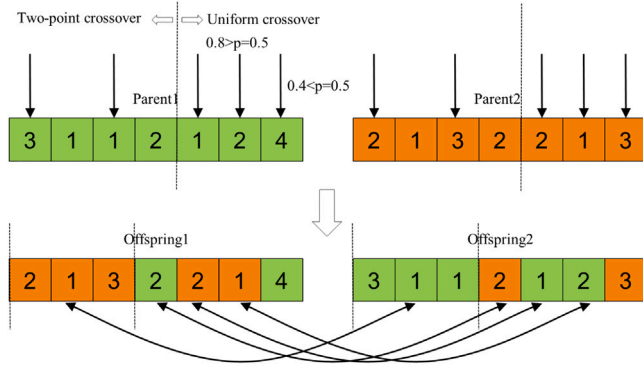


Fig. 6. Mixed crossover.

for selecting actions that are suitable for the current state, enabling the GA to adapt dynamically and iteratively tune its strategy to find better solutions.

3.4. Crossover operation

In the GA, the crossover operation is a key mechanism for generating new individuals. In this paper, we employed four different crossover methods to perform gene exchange in the *MS* of the parent individuals, and used the partially matched crossover method in the *OS*.

- **Two-point Crossover** (C_{tp}): Two-point crossover is randomly selected within the *MS* of the parent chromosomes, and the genes between these two points are swapped.
- **Multi-point Crossover** (C_{mp}): Similar to two-point crossover, but with multiple crossover points for gene swapping. It is particularly suited for the mid-to-late stages of the algorithm, helping to avoid local optima.
- **Uniform Crossover** (C_u): The uniform crossover method exchanges genes in the *MS* of the parent chromosomes bit by bit according to a predefined crossover probability. During the early iterations of GA, frequent gene exchanges can produce a wider variety of gene combinations.
- **Mixed Crossover** (C_m): The mixed crossover method combines the advantages of two-point crossover and uniform crossover by dividing the genes in the *MS* into two segments: one segment undergoes two-point crossover, while the other undergoes uniform crossover. This method retains the effective local optimization capability of two-point crossover while incorporating the broad search characteristics of uniform crossover. The specific crossover process is shown in Fig. 6.

For the *OS*, we employ the partially matched crossover method. An integer $r \in \mathbb{Z}_{\geq 0}$ is randomly generated, and r operation positions are selected. The operation sequence from one parent is copied to the offspring at the selected positions, while the remaining operations are filled based on the order from the other parent, ensuring genetic diversity and the validity of the offspring chromosomes.

By selecting appropriate crossover operations for the *MS* at different stages of the GA and combining them with the crossover method for the *OS*, the convergence results can be improved.

Remark 4. The four crossover methods vary in range and intensity, making them suitable for different GA stages. The mixed crossover method, for example, increases intensity in the early stages to accelerate population evolution. By flexibly combining these methods, the GA can update the population more effectively and converge faster to the optimal solution.

3.5. Mutation operation

In the GA, the mutation operation is a crucial mechanism for generating new genes and maintaining population diversity. By introducing mutations, the algorithm can avoid premature convergence to local optima and help solutions escape from local traps. In this paper, we designed four mutation strategies for the *MS* of the FJSP.

- **Machine Selection Mutation Based on Processing Time** (M_{pt}): This mutation method selects the machine with the shortest processing time, identifies gene positions needing mutation, determines the corresponding job and operation, and replaces the original machine with the one having the shortest processing time. The algorithm pseudocode is shown in Algorithm 1. To introduce Algorithm 1, we first define the necessary variables:
 - **CHS**: $CHS \in \mathbb{Z}_{\geq 0}^{2 \times T_0}$ represents the current solution, where T_0 is the total number of operations.
 - **O**: $O \in \mathbb{R}_{\geq 0}^{I \times J \times K}$ is the processing time matrix, where I represents jobs, J represents operations, and K represents machines.
 - **J**: J is a dictionary $J : \{k : v \mid k, v \in \mathbb{Z}_{\geq 0}\}$, where k is the job index and v is the number of operations for each job.

The set of possible machines for a given operation is defined as:

$$p_m = \{k \mid O_{ijk} > 0 \text{ and } k \neq M_{\text{num}}\} \quad (14)$$

where O_{ijk} represents the processing time for job i , operation j , on machine k , $O_{ijk} > 0$ ensures that machine k can perform the operation, and $k \neq M_{\text{num}}$ excludes the currently assigned machine.

- **Machine Selection Mutation based on Maximum Machine Load** (M_{mml}): This mutation method for the *MS* is based on maximum machine load. It calculates each machine's utilization rate, selects the highest one, randomly chooses an operation from it, and reassigns it to a machine with a lower load capable of processing it. The algorithm pseudocode is shown in Algorithm 2. To calculate the utilization U_m of each machine m , we use the following formula:

$$U_m = \frac{T_{a,m}}{f^*} \quad \forall m \in \mathcal{M} \quad (15)$$

where $T_{a,m}$ represents the active processing time of machine m , f^* denotes the best fitness of current population, and $\mathcal{M}$ is the set of all machines.

- **Machine Selection Mutation Based on Average Machine Load** (M_{aml}): This method achieves mutation by reallocating operations from machines with loads above the average to machines with loads below the average, as shown in the pseudocode in Algorithm 3. the average utilization across all machines is calculated as follows:

$$U_{\text{avg}} = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} U_m \quad (16)$$

where U_m from Eq. (15), and $|\mathcal{M}| \in \mathbb{Z}_{\geq 0}$ is the total number of machines.

- **Machine Selection Mutation Based on Critical Path** (M_{cp}): This mutation method optimizes the schedule by adjusting operations on the critical path, defined as the set of operations assigned to bottleneck machines. It involves reassigning each operation on the critical path to a machine with a lower load to alleviate bottleneck issues. The specific mutation process is shown in Fig. 7.

Algorithm 1: Machine Selection Mutation Based on Processing Time**Input :** CHS, O, T_o, J **Output:** New solution

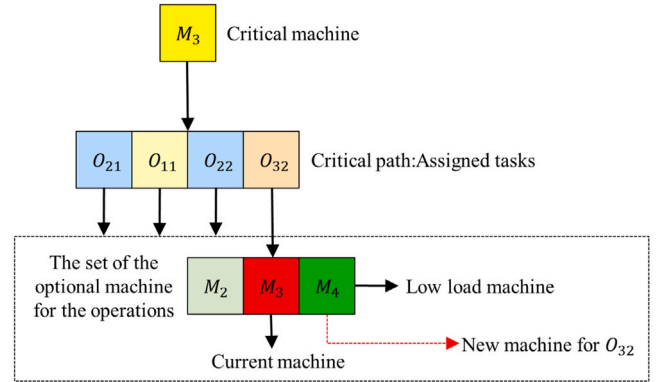
$T_r \leftarrow [0, 1, \dots, T_o - 1]$. $\triangleright$ All operation indices
 $MS \leftarrow [CHS_0, CHS_1, CHS_2, \dots, CHS_{T_o-1}]$. $\triangleright$ Extract MS from CHS

Choose an arbitrary r such that $1 \leq r < T_o$. $\triangleright$ Number of mutations

Reorder the elements in T_r . $T_r \leftarrow [T_{r0}, T_{r1}, \dots, T_{r(r-1)}]$. $\triangleright$ Select the operations to mutate from reordered list $l \leftarrow 0$.**while** $l < r$ **do** $num \leftarrow T_r[l]$. $T_o \leftarrow [0, 1, \dots, T_o - 1]$. $K \leftarrow []$. $Site \leftarrow 0$. **foreach** (k, v) **in** J **do** Add the slice $[T_o(Site), \dots, T_o(Site+v)]$ to K . $Site \leftarrow Site + v$. **end** **for** $i \in [0, \text{len}(K)]$ **do** **if** $num \in K[i]$ **then** $j \leftarrow i$. $\triangleright$ Job index Let o be the position of num in $K[i]$. $\triangleright$ Operation index **break**. **end** **end** Get the p_m from Eq. (14). $\triangleright$ Machines for operation (j, o)

$m_n \leftarrow \arg \min_{k \in p_m} O_{j,o,k}$. $\triangleright$ Select machine with minimum processing time
 Set $MS[\text{position of } num \text{ in the sequence}]$ to m_n . $\triangleright$ Update machine selection for num

$l \leftarrow l + 1$.

endCombine MS and OS to obtain CHS .**return** CHS .**Algorithm 2:** Machine Selection Mutation Based on Maximum Machine Load**Input :** CHS, O, T_o, J **Output:** New solution $MS \leftarrow [CHS_0, CHS_1, CHS_2, \dots, CHS_{T_o-1}]$.Calculate U_m from Eq. (15). $\triangleright$ Compute utilization for each machine $m^* \leftarrow \arg \max_{m \in \mathcal{M}} (U_m)$. $\triangleright$ Find the machine with max utilization**if** m^* has assigned tasks **then** Select a random task t_s from m^* . Get p_m of t_s from Eq. (14). $\triangleright$ Find machines that can process t_s Randomly select a machine m_n from p_m . Set $MS[\text{position of } t_s \text{ in the sequence}]$ to m_n . $\triangleright$ Update machine selection for t_s **end**Combine MS and OS to obtain CHS .**return** MS .**Algorithm 3:** Machine Selection Mutation Based on Average Machine Load**Input :** CHS, O, T_o, J **Output:** New solution $MS \leftarrow [CHS_0, CHS_1, CHS_2, \dots, CHS_{T_o-1}]$.Calculate U_{avg} from Eq. (16). $\mathcal{M}_h \leftarrow \{m \in \mathcal{M} \mid U_m > U_{avg}\}$. $\triangleright$ High load machines $\mathcal{M}_l \leftarrow \{m \in \mathcal{M} \mid U_m < U_{avg}\}$. $\triangleright$ Low load machines**if** $\mathcal{M}_h \neq \emptyset$ **and** $\mathcal{M}_l \neq \emptyset$ **then** Randomly select a high-load machine m_h from $\mathcal{M}_h$. Randomly select a task t_s from m_h . Get p_m of t_s from Eq. (14). $p_l \leftarrow \{m \in p_m \mid U_m < U_{avg}\}$. $\triangleright$ Low-utilization capable machines **if** $p_l \neq \emptyset$ **then** Select an arbitrary machine m_n from p_l . **end** **else** Select an arbitrary machine m_n from p_m . **end** Set $MS[\text{position of } t_s \text{ in the sequence}]$ to m_n . $\triangleright$ Update machine selection for t_s **end**Combine MS and OS to obtain CHS .**return** CHS .Fig. 7. MS mutation based on critical path.

By applying different mutation strategies at different stages of GA iteration, search performance and solution diversity can be more effectively improved, reducing the possibility of getting stuck in local optima.

Remark 5. These four mutation methods are specifically designed for the FJSP, leveraging processing time and machine load information to perform mutation operations. Compared to general mutation methods, these targeted approaches can effectively reduce the load on bottleneck machines, improve solution quality, and enhance overall scheduling efficiency. Moreover, the different mutation methods are suitable for various stages of the GA and instances of different scales. Flexibly applying these methods helps to improve the search efficiency of the GA.

3.6. Action

In this paper, the action space is defined across four key aspects: crossover rate, mutation rate, crossover operator, and mutation operator.

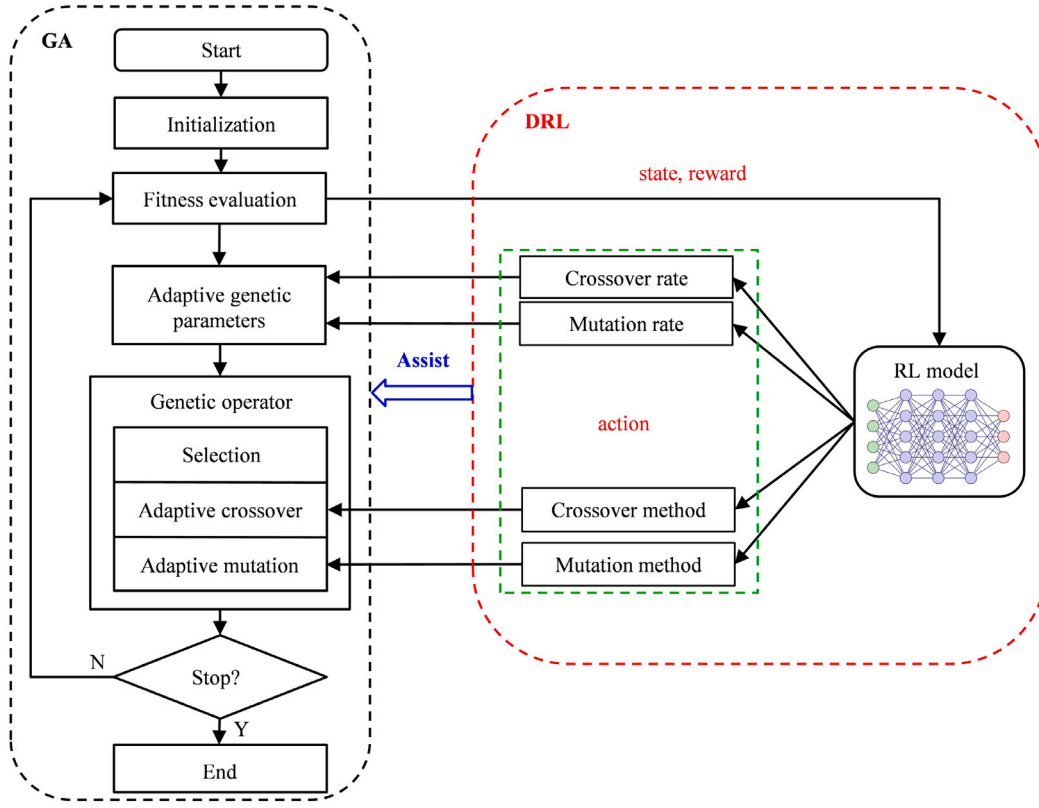


Fig. 8. The structure diagram of DRL-A-GA.

Specifically, the selectable ranges for the crossover rate are: {0.8, 0.9, 1.0}. For the mutation rate, they are: {0.1, 0.2, 0.3}. The crossover operations include four types: $\{C_{lp}, C_{mp}, C_u, C_m\}$. Similarly, mutation operations include four types: $\{M_{pt}, M_{mm}, M_{aml}, M_{cp}\}$.

Each action is composed of a combination of these four parameters. By selecting different actions, the agent can dynamically choose the optimal hyperparameter configuration and evolutionary operation for the GA.

3.7. Reward

In the DRL framework, the design of the reward function plays a crucial role in guiding the agent's learning process. In this paper, we define a reward function based on a local optimum l_{opt} to evaluate the agent's performance in solving the FJSP. Here, l_{opt} represents a known local optimum for a specific instance, typically derived from historical data or prior knowledge, and may vary across different instances.

The formula for the reward function is presented as follows:

$$r = \begin{cases} l_{opt} - f^*, & \text{if } l_{opt} < f^* \\ \lambda(l_{opt} - f^*), & \text{if } l_{opt} \geq f^* \end{cases} \quad (17)$$

Thus, if the best fitness $f^* > l_{opt}$, the agent receives a negative reward, calculated as $l_{opt} - f_{it_{best}}$. This implies that when the solution deviates from or exceeds the local optimum, the agent receives negative feedback, encouraging it to search for better solutions in the solution space.

Conversely, when the current solution's best fitness $f^* \leq l_{opt}$, the agent is given a positive reward to motivate further exploration towards superior solutions. To enhance this positive incentive, we introduce a scaling factor λ , which is tuned according to different instances. By incorporating the scaling factor, we significantly increase the positive reward when the solution surpasses the local optimum, driving the agent to explore the optimal solution space more aggressively.

Remark 6. Dynamically selecting appropriate parameters and evolutionary operators based on the current population state can effectively improve population quality, thereby enabling the GA to more efficiently find high-quality solutions. This reward function design effectively balances positive and negative feedback, encouraging the agent to seek solutions that surpass the local optimum.

3.8. Environment

In the DRL-A-GA, the entire iteration process of GA is regarded as the environment for DRL. The environment's main tasks are generating DRL state information, receiving and executing DRL actions, updating the GA state accordingly, and calculating reward values. At the end of each iteration, the reward is based on the difference between the current population's best fitness and the pre-set local optimum.

Remark 7. By treating the GA iteration process as the environment, the DRL-A-GA framework enables dynamic feedback on GA's evolving state. This setup allows the DRL agent to make informed adjustments, with rewards calculated from the population's performance, ultimately driving the GA achieves a more optimal solution.

3.9. Learning strategy

In this paper, we apply the double DQN (DDQN) algorithm as the core strategy for RL. This algorithm, proposed by Hasselt et al. (2016), is designed to address the issue of Q -value overestimation in DQN when handling high-dimensional state spaces. In DDQN, the Q -value update formula is as follows:

$$Q(s_t, a_t; \theta) \leftarrow Q(s_t, a_t; \theta) + \alpha [r_t + \gamma Q(s_{t+1}, a^*; \theta^-) - Q(s_t, a_t; \theta)] \quad (18)$$

where the action a^* is selected by the online network:

$$a^* = \arg \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}; \theta) \quad (19)$$

Algorithm 4: DRL-A-GA Algorithm**Input :** Population size N , Max generations $G_{\max}$ **Output:** Optimized solution

```

 $\mathcal{P} \leftarrow \{p_1, p_2, \dots, p_N\}$ . ▷ Population initialization
Compute initial population fitness to obtain  $\bar{f}_{\text{init}}, f_{\text{init}}^*, f_{\text{init}}^{\text{var}}$ .
 $g = 0$ 
for  $g < G_{\max}$  do
     $c_r, m_r, c_o, m_o \leftarrow \pi(s_n)$ . ▷ Agent chooses action based on state  $s_n$ 
    for each selected  $p_i$  in  $\mathcal{P}$  do
         $C_{\text{after}} \leftarrow \emptyset$ . ▷ Initialize offspring container
        Randomly select a number  $n_c \in (0, 1)$ .
        Randomly select a number  $n_m \in (0, 1)$ .
        if  $n_c \leq c_r$  then
            Randomly select an individual  $p_j$  from  $\mathcal{P}$ .
             $C_n \leftarrow c_o(p_i, p_j)$ . ▷ Apply crossover  $c_o$ 
            Add offspring  $C_n$  to  $C_{\text{after}}$ .
            Add parent  $p_i$  to  $C_{\text{after}}$ .
        end
        if  $n_m \leq m_r$  then
             $M_n \leftarrow m_o(C_n)$ . ▷ Apply mutation  $m_o$ 
            Add offspring  $M_n$  to  $C_{\text{after}}$ .
        end
        if  $C_{\text{after}} \neq \emptyset$  then
             $p_i \leftarrow \arg \min_{p \in C_{\text{after}}} \text{fitness}(p)$ . ▷ Replace with the parent individual
        end
    end
    Update state  $s_n$  using Eqs. (10)-(13).
    Calculate reward  $R$  from Eq. (17).
    Update policy( $\pi$ ) with Eqs. (18)-(20).
     $g \leftarrow g + 1$ .
end
return Optimized solution.

```

and the target Q -value is computed by the target network:

$$Q(s_{t+1}, a^*; \theta^-) \quad (20)$$

By separating action selection from action evaluation, DDQN can more accurately estimate the value of state–action pairs, reducing the risk of Q -value overestimation. Therefore, this paper employs DDQN as a learning strategy to assist the GA in parameter optimization and decision-making.

The pseudocode is provided in Algorithm 4, clearly outlining the overall process and each step of the DRL-A-GA algorithm. In addition, Fig. 8 presents the structure diagram of DRL-A-GA, offering a visual representation of the framework and clearly illustrating the relationship between DRL and GA.

4. Experiment results and discussion

In this Section, we show the experimental results. To evaluate the effectiveness and performance of the proposed algorithmic framework, we selected three distinct sets of benchmark data instances. The first set includes 4 small-scale FJSP instances from Kacem et al. (2002), the second set contains 10 large-scale FJSP instances from Brandimarte (1993), and the third set consists of 20 instances from Fattahi et al. (2007). To demonstrate the proposed algorithm's validity, we compared it with traditional optimization algorithms and state-of-the-art hybrid optimization algorithms. Additionally, to assess the contribution and impact of each component within the framework, we conducted ablation experiments by removing or replacing certain components to analyze their effect on overall performance.

4.1. Experimental settings

All experiments were conducted on a PC with the same configuration to ensure comparability and consistency of the results, equipped with an Intel core i9-14900KF processor (3.20 GHz) and 16 GB of RAM. The GPU is NVIDIA GeForce RTX 4080 Super. The experiments were implemented using python 3.8.19 as the programming language, and the deep learning framework used was pytorch version 2.3.0.

In this paper, the population size for the GA was set to 300, with a maximum of 200 iterations. The crossover and mutation probabilities, as well as the specific crossover and mutation strategies, were dynamically selected by the DRL model. For the DDQN, the learning rate was set to 0.0001, the discount factor to 0.98, the replay buffer size to 40,000, the batch size to 64, and the target network update frequency to 50 steps.

4.2. Experimental comparison

4.2.1. Comparison between DRL-A-GA and other traditional optimization algorithms

In order to validate the effectiveness of the proposed DRL-A-GA algorithm for solving the FJSP, we compared it with several classical optimization algorithms. These algorithms include Kacem's GA (Kacem et al., 2002), TS (Brandimarte, 1993), PSO (Ding and Gu, 2020b), HA (Sutton and Barto, 1998), GWO (Jiang and Zhang, 2018), and KBACO (Xing et al., 2010). Table 5 presents the best makespan values obtained by each algorithm on 10 benchmark instances (MK01-MK10) from the Brandimarte dataset. These values represent the optimal solutions obtained after running 20 experiments for each algorithm on each instance. Fig. 9 shows the relative percentage deviation (RPD) values for each algorithm, which are used to assess the deviation of each algorithm's solution from the known optimal solutions.

The RPD metric is used to evaluate the optimization performance and stability of the algorithms and is defined as follows:

$$RPD = \frac{BM - LB}{LB} \times 100\% \quad (21)$$

Where BM represents the best makespan by the algorithm, and LB represents the known best value. A smaller RPD value indicates a higher quality of the solution with a smaller deviation from the known optimal solution. Thus, RPD effectively reflects the optimization performance and stability of an algorithm across different instances.

From Table 5, the DRL-A-GA algorithm outperforms other classical algorithms on 7 out of the 10 benchmark instances (MK02, MK04, MK05, MK06, MK07, MK09, MK10). It shows particularly significant advantages on instances MK02, MK04, MK06, and MK10, highlighting its superior performance in solving the FJSP. In contrast, the performance of other algorithms varies considerably across different instances. For example, the KBACO algorithm performs better than the others on the MK01 instance, while on instances MK03 and MK08, most algorithms perform similarly.

Fig. 9 presents the boxplot of RPD values for each algorithm. It can be observed that the box height for the DRL-A-GA algorithm is significantly lower, indicating that its RPD values are more tightly clustered with smaller variance, demonstrating higher stability. The red line inside the box represents the median, and for DRL-A-GA, this median is positioned at a lower point, indicating that the algorithm consistently converges to better solutions.

Additionally, the green diamond in the box represents the mean RPD value. For DRL-A-GA, the green diamond is obviously lower than those of the other algorithms, further confirming that DRL-A-GA is capable of finding solutions closer to the optimal.

The open circles in the figure represent outliers, indicating extreme RPD values that fall outside the expected range. It is apparent that the outliers are more remarkable for algorithms like K'GA and TS, suggesting that these algorithms exhibit greater performance fluctuations and

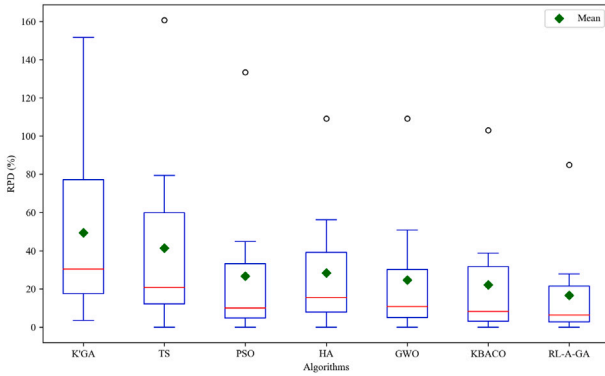
Table 5

Comparison of the makespan between DRL-A-GA and other traditional optimization algorithms on Brandimarte benchmark.

Instance	$n \times m$	Scale	LB	K'GA	TS	PSO	HA	GWO	KBACO	DRL-A-GA
MK01	10×6	55	36	44	42	40	42	40	39	40
MK02	10×6	58	24	38	32	29	28	29	29	26
MK03	15×8	150	204	225	211	204	204	204	204	204
MK04	15×8	90	48	88	81	66	75	64	65	60
MK05	15×4	106	168	195	186	175	179	175	173	173
MK06	10×15	150	33	83	86	77	69	69	67	61
MK07	20×5	100	133	178	157	145	149	147	144	139
MK08	20×10	225	523	542	523	523	555	523	523	523
MK09	20×10	240	299	380	369	320	342	322	311	307
MK10	20×15	240	165	311	296	239	242	249	229	211

Table 6The test of normality and homogeneity of variance (the level of significant $\alpha = 0.05$).

		K'GA	TS	PSO	HA	GWO	KBACO	DRL-A-GA
Normality	Statistic	0.867	0.788	0.681	0.770	0.745	0.722	0.675
	df	10	10	10	10	10	10	10
	Significant	0.092	0.010	0.001	0.006	0.003	0.002	0.000
Homogeneity of variance	Statistic	df_1		df_2		Significant		
	0.904	6		63		0.498		

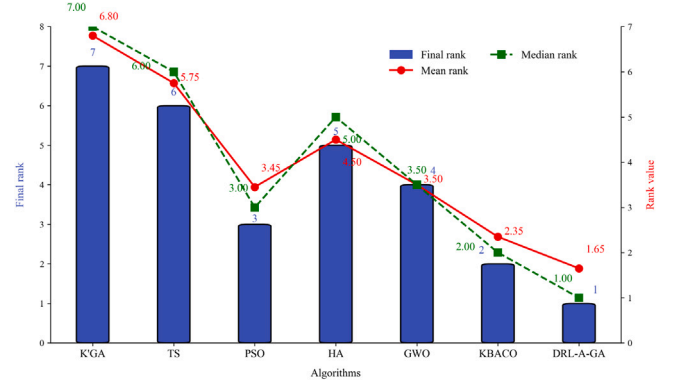
**Fig. 9.** Boxplot of RPD for traditional algorithms and DRL-A-GA.

lower stability. Overall, DRL-A-GA not only demonstrates good stability but also outperforms other algorithms in terms of solution quality.

To compare the significant differences between the proposed DRL-A-GA algorithm and other traditional optimization algorithms in terms of RPD values, a significance test was performed on the RPD values of all algorithms. Before conducting the significance test, Shapiro–Wilk normality test and one-way Anova variance homogeneity test were conducted at a significance level of $\alpha = 0.05$, as shown in Table 6. In the normality test, the *statistic* and *significant* values were used to determine whether the data conforms to a normal distribution. The results showed that except for the K'GA algorithm, the RPD values of other algorithms did not conform to a normal distribution. In the variance homogeneity test, the RPD values of all algorithms showed homogeneity (*homogeneity of variance*).

Due to the fact that most data in practice is not subject to normal distribution, the Friedman test (Cheraghalipour et al., 2018), a non-parametric test suitable for data with non-normal distribution or heterogeneous variances, was used for significance testing. Fig. 10 further shows the final ranks, mean ranks, and median ranks of several algorithms when solving the problem. It can be seen that DRL-A-GA performs the best, with the final rank of 1, the mean rank of 1.65, and the median rank of 1.00, significantly outperforming other algorithms. The median rank reflects the stability of the algorithm across multiple runs, indicating the stability and consistency of the DRL-A-GA algorithm.

To further compare the differences between the proposed algorithm and other algorithms, we used the Friedman test to verify its statistical

**Fig. 10.** Comparison of ranks between traditional algorithms and DRL-A-GA.**Table 7**The results of Friedman statistic χ^2 in different significance levels.

k	df	α	χ^2	$\chi^2_{\alpha(k-1)}$	H_0	H_1
7	6	0.05	42.750	12.592	×	✓
7	6	0.01	42.750	16.812	×	✓

significance. Table 7 shows the Friedman statistic χ^2 test results at different significance levels. For the number of groups k is 7, the degree of freedom df is 6, when the significance level $\alpha = 0.05$, the calculated Friedman statistic χ^2 is 42.750, and the critical value $\chi^2_{\alpha(k-1)}$ obtained from the table is 12.592. Since $\chi^2 = 42.750 > \chi^2_{\alpha(k-1)} = 12.592$, the null hypothesis (H_0) is rejected, and the alternative hypothesis (H_1) is accepted. Similarly, when the significance level $\alpha = 0.01$, $\chi^2 = 42.750 > \chi^2_{\alpha(k-1)} = 16.812$, the null hypothesis (H_0) is also rejected. This indicates that there are significant differences in algorithm performance at significance levels $\alpha = 0.05$ and 0.01.

Furthermore, Table 8 presents the Friedman statistic F_{ID} test results at different significance levels. For $k = 7$, $df = 6$, when the significance level $\alpha = 0.05$, the calculated F_{ID} value is 22.304, and the F critical value $F_{(k-1),(k-1)(n-1)}$ is 2.272. Since $F_{ID} = 22.304 > F_{(k-1),(k-1)(n-1)} = 2.272$, the null hypothesis (H_0) is rejected. Similarly, at the significance level $\alpha = 0.01$, $F_{ID} = 22.304 > F_{(k-1),(k-1)(n-1)} = 3.156$, the null hypothesis (H_0) is still rejected. These results further verify that at significance levels $\alpha = 0.05$ and 0.01, there are significant differences in the RPD rankings among the 7 algorithms being compared.

Table 8The results of Friedman statistic F_{ID} in different significance levels.

k	df	α	F_{ID}	$F_{(k-1),(k-1)(n-1)}$	H_0	H_1
7	6	0.05	22.304	2.272	×	✓
7	6	0.01	22.304	3.156	×	✓

Table 9

Comparison of the makespan between DRL-A-GA and other hybrid optimization algorithms on Brandimarte benchmark.

Instance	MAT SPSO	HG WO	HLO- PSO	SL GA	ISL GA	HGA- VNS	DRL- A-GA
MK01	39	40	40	40	40	40	40
MK02	27	29	28	27	27	26	26
MK03	207	204	204	204	204	204	204
MK04	65	65	63	60	64	60	60
MK05	174	175	175	172	173	173	173
MK06	72	79	71	69	63	61	61
MK07	154	149	144	144	140	140	139
MK08	523	523	523	523	523	523	523
MK09	340	325	326	320	307	307	307
MK10	299	253	238	254	218	214	211

These results indicate that the DRL-A-GA algorithm significantly outperforms the 6 traditional optimization algorithms being compared.

4.2.2. Comparison between DRL-A-GA and other hybrid optimization algorithms

To further validate the performance of the DRL-A-GA algorithm in solving the FJSP, we compared it with several advanced hybrid optimization algorithms that represent the current state-of-the-art methods, including MATSPSO (Henchiri and Ennigrou, 2013), HGWO (Tianhua, 2018), HLO-PSO (Ding and Gu, 2020a), SLGA (Chen et al., 2020), ISLGA (Jiang et al., 2023), and HGA-VNS (Sun et al., 2023). Tables 9 and 10 present the optimal makespan values and RPD values of these algorithms on the ten Brandimarte benchmark instances, respectively.

As shown in Table 9, the DRL-A-GA algorithm demonstrates outstanding performance across the ten benchmark instances, particularly achieving the best makespan values in six instances: MK02, MK04, MK06, MK07, MK09 and MK10. In contrast, although other advanced algorithms exhibit superior performance in certain specific instances, such as MATPSO and HGA-VNS obtaining better results in MK01 and MK09, their overall performance across most instances is less stable and comprehensive compared to the DRL-A-GA algorithm.

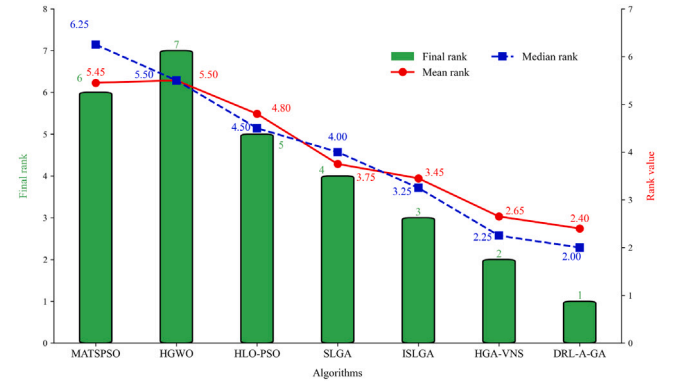
The RPD results in Table 10 further confirm this observation. The DRL-A-GA algorithm consistently achieves the lowest RPD values in the majority of instances, with an average RPD of only 16.73%, the lowest among all the algorithms compared. This indicates that the DRL-A-GA algorithm not only finds better solutions when solving the FJSP but also produces solutions closer to the known optimal ones. This stability and precision are particularly notable in instances such as MK07 and MK10, where the RPD values of DRL-A-GA are significantly lower than those of other algorithms, demonstrating its superior performance in solving these instances. These experimental results indicate that the DRL-A-GA algorithm outperforms many advanced hybrid optimization algorithms in terms of solution quality and stability.

To compare the DRL-A-GA algorithm proposed in this paper with other hybrid optimization algorithms in terms of RPD values, we conducted significance tests on the RPD values of all the algorithms. The testing procedures are consistent with those described in the previous section for classical optimization algorithms.

The Shapiro-Wilk normality tests and one-way Anova homogeneity of variance tests were performed, with the significance level set at $\alpha = 0.05$. The results of the normality test are shown in Table 11, indicating that the RPD values of all algorithms do not follow a normal distribution. However, the homogeneity of variance test showed that the RPD values of all algorithms exhibit homogeneity.

Table 10Comparison of the RPD (%) between DRL-A-GA and other hybrid optimization algorithms on Brandimarte benchmark.

Instance	MAT SPSO	HG WO	HLO- PSO	SL GA	ISL GA	HGA- VNS	DRL- A-GA
MK01	8.33	11.11	11.11	11.11	11.11	11.11	11.11
MK02	12.50	20.83	16.67	12.50	12.50	8.33	8.33
MK03	1.47	0	0	0	0	0	0
MK04	35.42	35.42	31.25	25.00	33.33	25.00	25.00
MK05	3.57	4.17	4.17	2.38	2.98	2.98	2.98
MK06	118.18	139.39	115.15	109.09	90.91	84.85	84.85
MK07	15.79	12.03	8.27	8.27	5.26	5.26	4.51
MK08	0	0	0	0	0	0	0
MK09	13.71	8.70	9.03	7.02	2.68	2.68	2.68
MK10	81.21	53.33	44.24	53.94	32.12	29.70	27.88
Mean	29.02	28.50	23.99	22.93	19.09	16.99	16.73

**Fig. 11.** Comparison of ranks between hybrid algorithms and DRL-A-GA.

Given that the data do not meet the assumption of normal distribution, the Friedman test was employed in this study. Table 12 presents the Friedman ranks, mean ranks, and median ranks of all hybrid optimization algorithms for each test instance. It can be observed that the DRL-A-GA algorithm performs excellently in most test instances, outpacing other hybrid optimization algorithms. Fig. 11 further illustrates the final ranks, mean ranks, and median ranks of several hybrid optimization algorithms when solving the problem. The results show that the DRL-A-GA algorithm ranks first in all tests, with the average rank of 2.40 and the median rank of 2.0, indicating that the algorithm significantly outperforms other hybrid optimization algorithms, demonstrating higher stability and consistency in performance.

Table 13 presents the results of the Friedman test statistic χ^2 at different significance levels. At significance levels $\alpha = 0.05$ and $\alpha = 0.01$, the calculated Friedman test statistic χ^2 values are significantly greater than the corresponding critical values $\chi^2_{\alpha(k-1)}$. Additionally, Table 14 shows the test results of the Friedman statistic F_{ID} . Similarly, at significance levels $\alpha = 0.05$ and $\alpha = 0.01$, the calculated F_{ID} values are also greater than the corresponding critical values $F_{(k-1),(k-1)(n-1)}$. This further confirms that there is a significant difference between the DRL-A-GA algorithm and other hybrid optimization algorithms in terms of RPD values.

To validate the efficiency of the proposed algorithm, we compared DRL-A-GA with several other optimization algorithms. The comparison includes the average runtime (each algorithm runs 20 times), makespan, and average RPD . The algorithms compared are ABC (Long et al., 2022), GWO (Jiang and Zhang, 2018), SLABC (Long et al., 2022), and HGWO (Tianhua, 2018), with all comparisons conducted on the Brandimarte benchmark. The experimental results are shown in Table 15. DRL-A-GA consistently demonstrated lower makespan values across all test instances, with an average RPD of 16.73%, the lowest among all algorithms. Furthermore, DRL-A-GA exhibited lower computational

Table 11The test of normality and homogeneity of variance (the level of significant $\alpha = 0.05$).

		MATSPSO	HGWO	HLO-PSO	SLGA	ISLGA	HGA-VNS	DRL-A-GA
Normality	Statistic	0.737	0.690	0.696	0.699	0.708	0.684	0.675
	df	10	10	10	10	10	10	10
	Significant	0.002	0.001	0.001	0.001	0.001	0.001	0.000
Homogeneity of variance		Statistic 0.459		df_1 6		df_2 63		Significant 0.836

Table 12

The Friedman rank of orthogonal experiments scheme.

Test Instance	MATS-PSO	H-GWO	HLO-PSO	SL-GA	ISL-GA	HGA-VNS	DRL-A-GA
MK01	1.0	4.5	4.5	4.5	4.5	4.5	4.5
MK02	4.0	7.0	6.0	4.0	4.0	1.5	1.5
MK03	7.0	3.5	3.5	3.5	3.5	3.5	3.5
MK04	6.5	6.5	4.0	2.0	5.0	2.0	2.0
MK05	5.0	6.5	6.5	1.0	3.0	3.0	3.0
MK06	6.0	7.0	5.0	4.0	3.0	1.5	1.5
MK07	7.0	6.0	4.5	4.5	2.5	2.5	1.0
MK08	4.0	4.0	4.0	4.0	4.0	4.0	4.0
MK09	7.0	5.0	6.0	4.0	2.0	2.0	2.0
MK10	7.0	5.0	4.0	6.0	3.0	2.0	1.0
Mean	5.45	5.50	4.80	3.75	3.45	2.65	2.40
Median	6.25	5.50	4.50	4.0	3.25	2.25	2.0

Table 13The results of Friedman statistic χ^2 in different significance levels.

k	df	α	χ^2	$\chi^2_{\alpha(k-1)}$	H_0	H_1
7	6	0.05	20.871	12.592	×	✓
7	6	0.01	20.871	16.812	×	✓

Table 14The results of Friedman statistic F_{ID} in different significance levels.

k	df	α	F_{ID}	$F_{(k-1),(k-1)(n-1)}$	H_0	H_1
7	6	0.05	4.801	2.272	×	✓
7	6	0.01	4.801	3.156	×	✓

times in several instances, with an overall runtime of 134.57 s, significantly lower than the other algorithms. These results indicate that DRL-A-GA not only provides high-quality solutions but also completes the computations in a shorter time, further confirming the efficiency and feasibility of the algorithm in solving the FJSP problem.

To further evaluate the stability of the algorithm in finding better solutions, we compared the average makespan of DRL-A-GA with several other optimization algorithms. The algorithms compared include LEGA (Ho et al., 2007), KBACO (Xing et al., 2010), IPSO (Ding and Gu, 2020b), HLO-PSO (Ding and Gu, 2020a), HQPSO (Zhang and Hu, 2019), HGWO (Tianhua, 2018), SLABC (Long et al., 2022), and SLGA (Chen et al., 2020). All comparisons were conducted on the Brandimarte benchmark. The experimental results are shown in the Table 16. The results indicate that DRL-A-GA demonstrates a lower average makespan in most of the test instances, particularly in MK02, MK04, MK05, MK06, MK07, MK09 and MK10, where its performance is significantly better than that of the other algorithms. This demonstrates that the proposed algorithm not only achieves high-quality solutions in individual experiments but also consistently maintains a high level of solution quality across multiple runs, thereby validating its stability.

To validate the generalization ability of the proposed algorithm, we compared DRL-A-GA with several other optimization algorithms (HSA/TS (Fattahi et al., 2007), HTS/SA (Fattahi et al., 2007), FPA (Phuang, 2017), MILP (Özgiiven et al., 2010), EPSO (Teekeng et al., 2016), AIA (Bagheri et al., 2010), GA (Zandieh et al., 2008), HSS (Külahlı et al., 2021), and IGAR (Amjad et al., 2021)) on the Fattahi dataset, focusing on the evaluation of makespan and average RPD . The experimental results, as shown in the Table 17, demonstrate

that DRL-A-GA achieves lower makespan in most of the test instances. Furthermore, due to the relatively simple nature of the SFJS series dataset, DRL-A-GA is able to consistently find solutions equal to the LB value. More importantly, DRL-A-GA's average RPD is 13.46%, the lowest among all the algorithms. These results further confirm that DRL-A-GA exhibits superior consistency across different datasets, validating its strong generalization capability.

Based on the results from Figs. 9–11 and Tables 5–17, it is evident that the proposed DRL-A-GA algorithm significantly outperforms several traditional and hybrid optimization algorithms when solving the FJSP.

4.2.3. Ablation experiment

Additionally, to verify the effectiveness and necessity of each component within the DRL-A-GA framework, we designed ablation experiments. These experiments focused on two specific strategies: the first strategy involved using DRL solely to assist in selecting key parameters of the GA, crossover and mutation rates. The second strategy involved using DRL solely to assist in selecting crossover and mutation methods. By comparing these two strategies with the complete DRL-A-GA algorithm and the conventional GA, we conducted a comprehensive analysis using multiple evaluation metrics, including the best makespan (BM), average makespan (AM), relative percentage deviation of best makespan (RPD_{BM}), relative percentage deviation of average makespan (RPD_{AM}) and the average runtime.

$$RPD_{BM} = \frac{BM - LB}{LB} \times 100\% \quad (22)$$

$$RPD_{AM} = \frac{AM - LB}{LB} \times 100\% \quad (23)$$

As shown in Table 18, the Kacem instance is relatively simple, allowing all four algorithms to achieve the same optimal solution in terms of BM , resulting in an obvious RPD_{BM} value of 0. However, in terms of AM , the DRL-A-GA algorithm shows a notable improvement, where DRL-A-GA further exhibits its superior optimization performance. Particularly in instances like 10×10 and 15×10 , DRL-A-GA achieves notably lower RPD_{AM} values compared to GA and other variants.

Table 19 presents the BM , AM , and average runtime (based on 20 runs) for different algorithms across multiple test instances. The results indicate that the DRL-A-GA algorithm consistently outperforms other ablation versions and traditional GA across all test instances. Specifically, in instances MK02, MK04, MK06, MK07, and MK10, the BM values of DRL-A-GA are significantly lower than those of other ablation algorithms, indicating superior performance in these cases. Moreover, in instances MK01 and MK05, the AM values of DRL-A-GA are very close to their corresponding BM , suggesting that the algorithm consistently finds the optimal solution in nearly every run, demonstrating a high level of stability and reliability.

However, to maintain the clear performance advantages of DRL-A-GA, its computational time is somewhat higher compared to other algorithms. This can be attributed to the increased complexity of DRL-A-GA, such as the use of reinforcement learning-based parameter and operator selection strategies, which require more computational steps and lead to longer runtimes.

It is worth noting that although DRL-A-GA's runtime is longer than that of some other ablation algorithms in certain instances, the increase in time is acceptable. In fact, in some test instances, DRL-A-GA even outperforms other ablation algorithms in terms of runtime. In

Table 15Comparison of the makespan, *RPD* (%) and computational time between DRL-A-GA and other algorithms on Brandimarte benchmark.

Test Instance	$n \times m$	Makespan					Time(second)				
		ABC	GWO	SLABC	HGWO	DRL-A-GA	ABC	GWO	SLABC	HGWO	DRL-A-GA
MK01	10 × 6	51	40	42	40	40	45.30	64.6	29.65	36.3	43.3
MK02	10 × 6	38	29	29	29	26	88.86	70.0	73.97	38.7	44.5
MK03	15 × 8	269	204	204	204	204	125.8	377.6	117.2	165.8	137.2
MK04	15 × 8	96	64	69	65	60	71.92	218.2	55.67	75.9	73.8
MK05	15 × 4	218	175	175	175	173	170.7	131.4	165.4	96.7	97.1
MK06	10 × 15	109	69	80	79	61	258.4	480.7	170.1	168.6	150.6
MK07	20 × 5	189	147	155	149	139	193.8	213.4	189.7	92.1	87.5
MK08	20 × 10	590	523	523	523	523	415.2	1026.2	366.3	340.8	228.5
MK09	20 × 10	465	322	368	325	307	536.3	1123.8	410.6	378.9	257.9
MK10	20 × 15	345	249	283	253	211	452.3	1744.3	332.7	388.5	225.3
Mean <i>RPD</i>		71.15	24.77	33.90	28.50	16.73	–	–	–	–	–
Mean Time		–	–	–	–	–	235.86	545.02	191.13	178.23	134.57

Table 16

Comparison of the average of makespan between DRL-A-GA and other algorithms on Brandimarte benchmark.

Instance	<i>LB</i>	LEGA	KBACO	IPSO	HLO-PSO	HQPSO	HGWO	SLABC	SLGA	DRL-A-GA
MK01	36	41.5	39.8	41.6	41.3	42.0	41.6	43.0	42.0	40.05
MK02	24	29.1	29.1	31.1	28.8	29.2	30.3	31.5	29.7	26.8
MK03	204	–	204.0	204.0	204.0	204.0	204.1	219.4	210.2	204.0
MK04	48	67.34	66.1	68.05	66.1	72.0	67.4	75.6	66.7	62.6
MK05	168	178.1	173.8	178.05	177.3	182.6	178.2	191.9	183.5	173.05
MK06	33	68.82	69.1	80.7	73.6	72.8	79.9	89.4	76.9	63.45
MK07	133	152.9	145.4	148.7	145.05	155.5	156.4	159.3	151.3	143.65
MK08	523	523.34	523.0	523.0	523.0	523.5	523.0	525.0	526.6	523.0
MK09	299	327.74	312.2	338.05	331.3	355.7	342.3	379.1	340.4	309.15
MK10	165	235.72	233.7	248.85	246.7	254.9	262.7	294.0	269.9	216.15

Note: The symbol “–” indicates that the corresponding value is not provided in the referenced paper.

Table 17Comparison of the makespan and mean *RPD* between DRL-A-GA and other optimization algorithms on the Fattahi dataset.

Instance	$n \times m$	<i>LB</i>	HSA/TS	HTS/SA	FPA	MILP	EPSO	AIA	GA	HSS	IGAR	DRL-A-GA
SFJS01	2 × 2	66	66	66	66	66	66	66	66	66	66	66
SFJS02	2 × 2	107	107	107	107	107	107	107	107	107	107	107
SFJS03	3 × 2	221	221	221	221	221	221	221	221	221	221	221
SFJS04	3 × 2	355	355	355	355	355	355	355	355	355	355	355
SFJS05	3 × 2	119	119	119	119	119	119	119	119	119	119	119
SFJS06	3 × 3	320	320	320	320	320	320	320	320	320	320	320
SFJS07	3 × 5	397	398	398	397	398	397	398	397	398	397	397
SFJS08	3 × 4	253	253	256	253	253	253	253	253	253	253	253
SFJS09	3 × 3	210	210	210	210	210	210	210	210	210	210	210
SFJS10	4 × 5	516	516	516	516	516	516	516	516	516	516	516
MFJS01	5 × 6	396	491	469	469	468	468	468	468	468	468	468
MFJS02	5 × 7	396	482	468	446	446	446	448	448	446	448	446
MFJS03	6 × 7	396	538	538	470	466	466	468	466	466	468	466
MFJS04	7 × 7	496	650	618	554	564	554	554	554	554	554	554
MFJS05	7 × 7	414	662	625	516	514	514	527	514	514	514	514
MFJS06	8 × 7	469	785	730	636	635	634	635	634	634	634	634
MFJS07	8 × 7	619	1081	947	879	935	879	879	881	879	881	879
MFJS08	9 × 8	619	1122	922	884	905	884	884	891	884	884	884
MFJS09	11 × 8	764	1243	1105	1062	1192	1059	1088	1094	1060	1097	1055
MFJS10	12 × 8	944	1615	1384	1221	1276	1205	1267	1286	1208	1275	1196
Mean <i>RPD</i>	–	–	26.49	19.92	13.74	15.52	13.53	14.27	14.29	13.56	14.22	13.46

Table 18Comparison of the makespan and *RPD* between DRL-A-GA and other ablation algorithms on Kacem benchmark.

Test Instance	<i>LB</i>	<i>BM</i>				<i>AM</i>				<i>RPD_{AM}</i>			
		GA	GA-Para	GA-Oper	DRL-A-GA	GA	GA-Para	GA-Oper	DRL-A-GA	GA	GA-Para	GA-Oper	DRL-A-GA
4 × 5	11	11	11	11	11	11.0	11.0	11.0	11.0	0	0	0	0
10 × 7	11	11	11	11	11	11.25	11.15	11.1	11.0	2.27	1.36	0.91	0
10 × 10	7	7	7	7	7	7.5	7.3	7.15	7.05	7.14	4.29	2.14	0.71
15 × 10	11	11	11	11	11	11.70	11.65	11.55	11.35	6.36	5.91	5.0	3.18
Mean	–	–	–	–	–	–	–	–	–	3.94	2.89	2.01	0.97

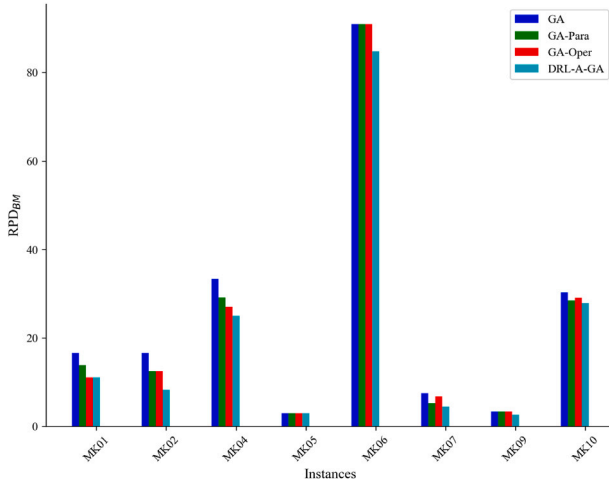
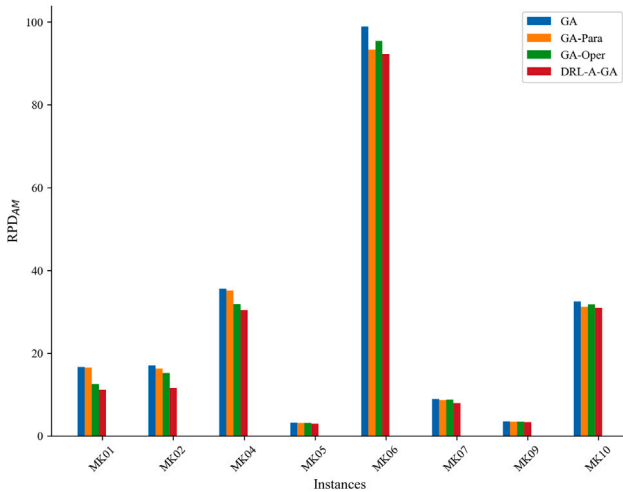
other words, while DRL-A-GA has slightly longer computation times, its advantages in algorithm performance and solution quality make it a worthwhile trade-off, ultimately enhancing the overall solution capability of the algorithm.

Figs. 12–13 further present the *RPD* of each algorithm on the test instances, including *RPD_{BM}* and *RPD_{AM}*. Since the values of *RPD_{AM}* and *RPD_{BM}* are both 0 for the MK03 and MK08 instances, the results for these two instances are not shown. The results of *RPD_{BM}* indicate

Table 19

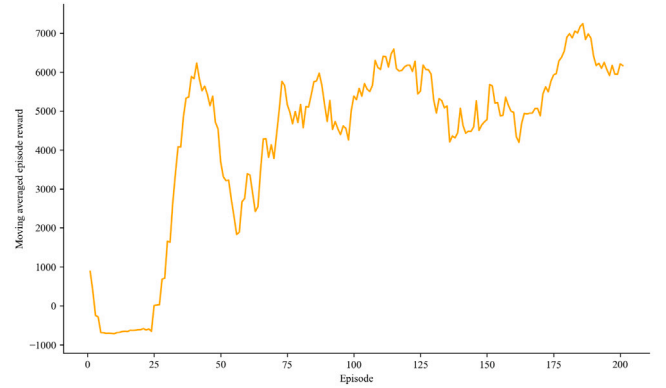
Comparison of the makespan and average runtime between DRL-A-GA and other ablation algorithms on Brandimarte benchmark.

Test	<i>LB</i>	<i>BM</i>				<i>AM</i>				Time (second)			
Instance		GA	GA-Para	GA-Oper	DRL-A-GA	GA	GA-Para	GA-Oper	DRL-A-GA	GA	GA-Para	GA-Oper	DRL-A-GA
MK01	36	42	41	40	40	42.0	41.95	40.55	40.05	35.17	41.74	42.04	43.31
MK02	24	28	27	27	26	28.1	27.9	27.65	26.8	36.99	43.95	43.30	44.45
MK03	204	204	204	204	204	204.0	204.0	204.0	204.0	110.42	142.02	129.70	137.16
MK04	48	64	62	61	60	65.1	64.9	63.3	62.6	59.05	79.61	70.16	73.75
MK05	168	173	173	173	173	173.45	173.4	173.4	173.05	72.89	96.41	87.28	97.10
MK06	33	63	63	63	61	65.65	63.8	64.5	63.45	114.52	148.66	136.08	150.61
MK07	133	143	140	142	139	144.95	144.55	144.75	143.65	68.54	84.25	81.04	87.50
MK08	523	523	523	523	523	523.0	523.0	523.0	523.0	182.94	213.43	214.17	228.48
MK09	299	309	309	309	307	309.6	309.4	309.35	309.15	187.89	251.66	223.69	257.88
MK10	165	215	212	213	211	218.6	216.5	217.45	216.15	185.20	233.02	219.50	225.39

**Fig. 12.** The RPD_{BM} of each algorithm.**Fig. 13.** The RPD_{AM} of each algorithm.

that DRL-A-GA consistently achieves the lowest RPD in most instances. For instance, in the MK06 instance, the RPD_{BM} for DRL-A-GA is 84.85%, which is significantly lower than that of other algorithms. Similarly, the DRL-A-GA also demonstrates superior performance in terms of RPD_{AM} , particularly in instances MK01, MK06, and MK10, indicating better stability.

These experimental results collectively demonstrate that the DRL-A-GA algorithm achieves its optimal performance only when DRL is employed to assist both in selecting the appropriate key parameters

**Fig. 14.** The learning curve of cumulative rewards for MK05.

(crossover and mutation rates) and in choosing suitable genetic operations for each iteration stage. Removing either DRL strategy significantly hampers the algorithm's ability to converge to optimal values. This confirms the effectiveness and irreplaceability of each component within the DRL-A-GA framework, underscoring that the comprehensive integration of these strategies is crucial for achieving optimal algorithm performance.

The Fig. 14 shows the reward variation trend during training. As training progresses, the rewards gradually rise from a low initial level. Despite some fluctuations, the overall trend indicates continuous improvement. Eventually, the rewards stabilize and fluctuate within a higher range, suggesting the algorithm's performance has become more stable and effective.

Fig. 15 presents the average convergence of four algorithms after running 20 times on the MK01, MK05, MK09, and MK10 instances. It can be observed that DRL-A-GA converges faster than the other algorithms in most cases and ultimately reaches better solutions.

To more intuitively demonstrate the optimization effectiveness of the proposed DRL-A-GA algorithm in solving the FJSP, we present improved scheduling schemes of several typical test instances in the form of Gantt charts. Figs. 16–18 clearly display the processing sequence and time allocation of each job on different machines, facilitating the observation of job start times, end times, and machine workload balancing.

4.3. Discussion

This paper employs DRL-A-GA to solve the FJSP and validates its effectiveness through extensive numerical experiments. The primary advantage of the proposed method lies in its ability to adaptively adjust the parameters of the genetic algorithm while leveraging reinforcement learning to select appropriate genetic operations for different evolutionary stages. This design enables dynamic adjustment of parameters

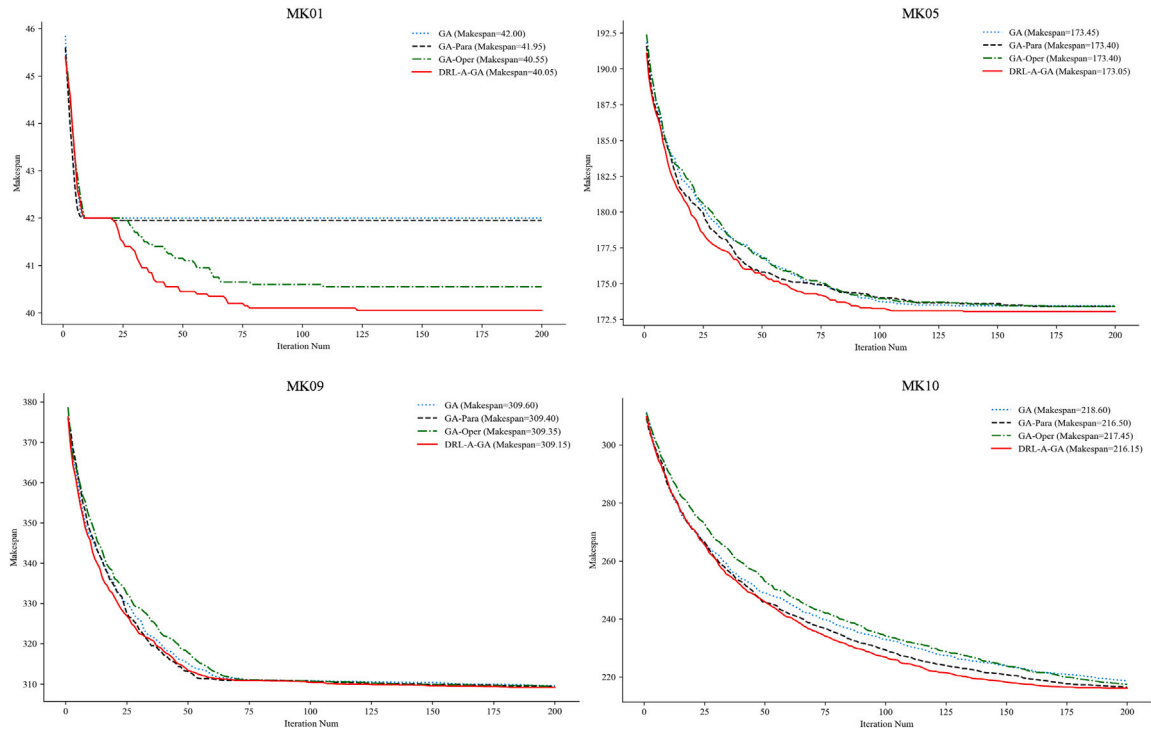


Fig. 15. The average convergence curves of convergence for MK01, MK05, MK09, MK10.

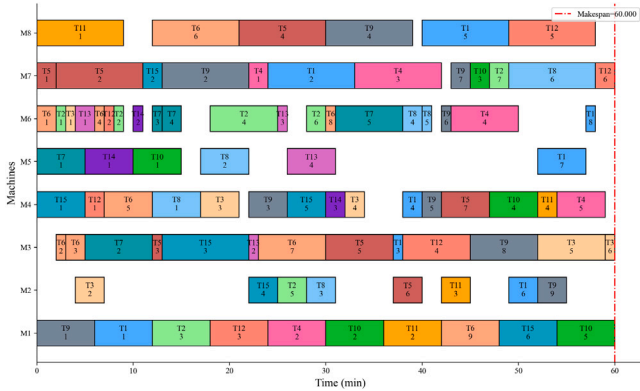


Fig. 16. Gantt chart of FJSP for MK04 using DRL-A-GA algorithm.

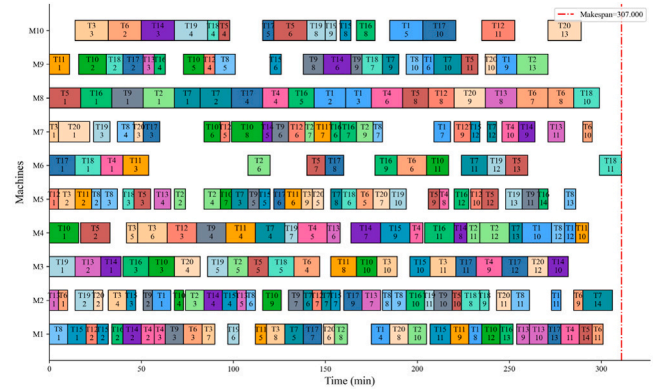


Fig. 18. Gantt chart of FJSP for MK09 using DRL-A-GA algorithm.

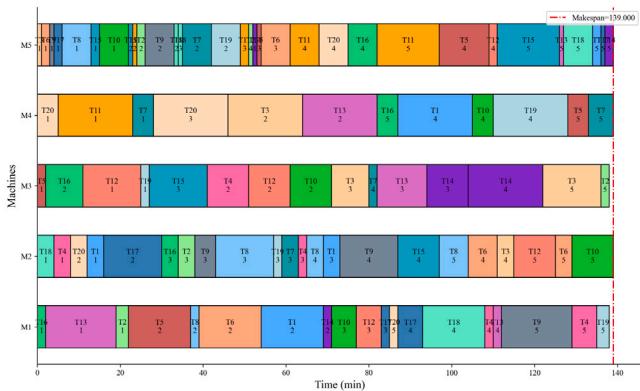


Fig. 17. Gantt chart of FJSP for MK07 using DRL-A-GA algorithm.

and operator combinations during the evolution process, significantly enhancing the efficiency and performance of population evolution.

However, there is still room for improvement in the algorithm. For instance, certain key steps and parameters of the genetic algorithm, such as the strategy for generating the initial population and the population size, are determined empirically. This reliance on experience may impact the efficiency of evolution. Moreover, although the integration of reinforcement learning improves the algorithm's intelligence, it may introduce additional hyperparameters, potentially increasing the total number of hyperparameters in the algorithm.

5. Conclusion and future work

As a classic optimization algorithm, GA has been widely used to solve FJSP. However, the performance of GA is significantly influenced by their key parameters and genetic operations. The final convergence of a GA to a satisfactory solution largely depends on the appropriateness of its parameter settings and operation choices. Therefore,

this paper proposes the DRL-A-GA algorithm to solve FJSP. To better represent the current state of the population in the GA, continuous state vectors are used. Furthermore, four mutation operations specifically designed for the FJSP problem are developed, and DRL is employed to select suitable genetic parameters and operations at different iteration stages, enhancing the algorithm's performance.

To validate the performance, efficiency, stability, and generalization ability of the proposed DRL-A-GA, we designed multiple sets of experiments to test and analyze the algorithm's performance. The proposed algorithm was compared with traditional classic optimization algorithms and advanced hybrid optimization algorithms, focusing on the comparison of makespan, *RPD*, average makespan, average runtime, and other metrics, followed by statistical analysis. Additionally, to assess the necessity of each component of the algorithm, we designed and conducted ablation experiments. The experimental results show that DRL-A-GA outperforms other algorithms in most test instances, confirming its superior optimization performance.

Future research focuses on improving the population initialization strategy by introducing RL assisted population initialization methods and combining with other optimization methods, such as PSO and SA, to construct a hybrid optimization framework.

CRediT authorship contribution statement

Jian Ma: Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Weinan Gao:** Writing – review & editing, Writing – original draft, Supervision, Resources, Methodology, Investigation. **Weitian Tong:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62373090.

Data availability

Data will be made available on request.

References

- Amjad, M.K., Butt, S.I., Anjum, N., 2021. Improved genetic algorithm integrated with scheduling rules for flexible job shop scheduling problems.
- Bagheri, A., Zandieh, M., Mahdavi, I., Yazdani, M., 2010. An artificial immune algorithm for the flexible job-shop scheduling problem. *Future Gener. Comput. Syst.* 26 (4), 533–541.
- Berrada, M., Steckel, K.E., 1986. A branch and bound approach for machine load balancing in flexible manufacturing systems. *Manag. Sci.* 32 (10), 1316–1335. <http://dx.doi.org/10.1287/mnsc.32.10.1316>.
- Brandimarte, P., 1993. Routing and scheduling in a flexible job shop by tabu search. *Ann. Oper. Res.* 41 (3), 157–183. <http://dx.doi.org/10.1007/BF02023073>.
- Chang, H.C., Liu, T.K., 2017. Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms. *J. Intell. Manuf.* 28 (8), 1973–1986. <http://dx.doi.org/10.1007/s10845-015-1084-y>.
- Chen, H., Ihlow, J., Lehmann, C., 1999. A genetic algorithm for flexible job-shop scheduling. *IEEE Int. Conf. Robot. Autom.* 2, 1120–1125. <http://dx.doi.org/10.1109/ROBOT.1999.772512>.
- Chen, R., Yang, B., Li, S., Wang, S., 2020. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Comput. Ind. Eng.* 149, 106778. <http://dx.doi.org/10.1016/j.cie.2020.106778>.
- Cheraghali, A., Hajiaghayi-Keshetli, M., Paydar, M.M., 2018. Tree Growth Algorithm (TGA): A novel approach for solving optimization problems. *Eng. Appl. Artif. Intell.* 72, 393–414. <http://dx.doi.org/10.1016/j.engappai.2018.04.021>.
- Ding, H., Gu, X., 2020a. Hybrid of human learning optimization algorithm and particle swarm optimization algorithm with scheduling strategies for the flexible job-shop scheduling problem. *Neurocomputing* 414, 313–332. <http://dx.doi.org/10.1016/j.neucom.2020.07.004>.
- Ding, H., Gu, X., 2020b. Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem. *Comput. Oper. Res.* 121, 104951. <http://dx.doi.org/10.1016/j.cor.2020.104951>.
- Fattahi, P., Mehrabad, M.S., Jolai, F., 2007. Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *J. Intell. Manuf.* 18 (3), 331–342. <http://dx.doi.org/10.1007/s10845-007-0026-8>.
- Garey, M.R., Johnson, D.S., Sethi, R., 1976. The complexity of flowshop and jobshop scheduling. *Math. Oper. Res.* 1 (2), 117–129. <http://dx.doi.org/10.1287/moor.1.2.117>.
- Gen, M., Gao, J., Lin, L., 2005. Multistage-based genetic algorithm for flexible job shop scheduling problem. *J. Complex. Int.* 48, 183–196. http://dx.doi.org/10.1007/978-3-540-95978-6_13.
- Gomes, M., Barbosa-Póvoa, A., Novais, A., 2005. Optimal scheduling for flexible job shop operation. *Int. J. Prod. Res.* 43 (11), 2323–2353. <http://dx.doi.org/10.1080/00207540412331330101>.
- Hasselt, H.v., Guez, A., Silver, D., 2016. Deep reinforcement learning with double Q-learning. In: Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence. AAAI '16, pp. 2094–2100. <http://dx.doi.org/10.48550/arXiv.1509.06461>.
- Henchiri, A., Ennigrou, M., 2013. Particle swarm optimization combined with tabu search in a multi-agent model for flexible job shop problem. In: Advances in Swarm Intelligence. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 385–394. http://dx.doi.org/10.1007/978-3-642-38715-9_46.
- Ho, N.B., Tay, J.C., Lai, E.M.-K., 2007. An effective architecture for learning and evolving flexible job-shop schedules. *European J. Oper. Res.* 179 (2), 316–333.
- Jeong, K.-C., Kim, Y.-D., 1998. A real-time scheduling mechanism for a flexible manufacturing system: Using simulation and dispatching rules. *Int. J. Prod. Res.* 36 (9), 2609–2626. <http://dx.doi.org/10.1080/002075498192733>.
- Jiang, M., Yu, H., Chen, J., 2023. Improved self-learning genetic algorithm for solving flexible job shop scheduling. *Mathematics* 11 (22), 4700.
- Jiang, T., Zhang, C., 2018. Application of grey wolf optimization for solving combinatorial problems: Job shop and flexible job shop scheduling cases. *IEEE Access* 6, 26231–26240. <http://dx.doi.org/10.1109/ACCESS.2018.2833552>.
- Kacem, I., Hammadi, S., Borne, P., 2002. Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Trans. Syst. Man Cybern. Part C (Appl. Rev.)* 32 (1), 1–13. <http://dx.doi.org/10.1109/TSMCC.2002.1009117>.
- Külahlı, S., Engin, O., Koç, İ., 2021. A new hybrid scatter search method for solving the flexible job shop scheduling problems.
- Liu, T.K., Tsai, J.T., Chou, J.H., 2006. Improved genetic algorithm for the job-shop scheduling problem. *Int. J. Adv. Manuf. Technol.* 27, 1021–1029. <http://dx.doi.org/10.1007/s00170-004-2283-4>.
- Long, X., Zhang, J., Qi, X., Xu, W., Jin, T., Zhou, K., 2022. A self-learning artificial bee colony algorithm based on reinforcement learning for a flexible job-shop scheduling problem. *Concurr. Comput.: Pr. Exp.* (4), 34.
- Meng, L., Zhang, C., Ren, Y., Zhang, B., Lv, C., 2020. Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Comput. Ind. Eng.* 142, 106347. <http://dx.doi.org/10.1016/j.cie.2020.106347>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.A., 2013. Playing atari with deep reinforcement learning. *Comput. Sci.* <http://dx.doi.org/10.48550/arXiv.1312.5602>.
- O'Keefe, R.M., Kasirajan, T., 1992. Interaction between dispatching and next station selection rules in a dedicated flexible manufacturing system. *Int. J. Prod. Res.* 30 (8), 1753–1772. <http://dx.doi.org/10.1080/00207549208948120>.
- Özgüven, C., Özbakır, L., Yavuz, Y., 2010. Mathematical models for job-shop scheduling problems with routing and process plan flexibility. *Appl. Math. Model.* 34 (6), 1539–1548. <http://dx.doi.org/10.1016/j.apm.2009.09.002>.
- Palmer, G.J., 1996. A simulated annealing approach to integrated production scheduling. *J. Intell. Manuf.* 7 (3), 163–176. <http://dx.doi.org/10.1007/BF00118077>.
- Pezzella, F., Morganti, G., Ciaschetti, G., 2008. A genetic algorithm for the Flexible Job-shop scheduling problem. *Comput. Oper. Res.* 35, 3202–3212. <http://dx.doi.org/10.1016/j.cor.2007.02.014>.
- Phuang, A., 2017. The flower pollination algorithm with disparity count process for scheduling problem. In: 2017 9th International Conference on Information Technology and Electrical Engineering. ICITEE, pp. 1–5.
- Shanker, K., Tzen, Y.J.J., 1985. A loading and dispatching problem in a random flexible manufacturing system. *Int. J. Prod. Res.* 23 (3), 579–595. <http://dx.doi.org/10.1080/00207548508904730>.
- Song, Y., Ou, J., Suganthan, P.N., Pedrycz, W., Yang, Q., Xing, L., 2023a. Learning adaptive genetic algorithm for earth electromagnetic satellite scheduling. *IEEE Trans. Aerosp. Electron. Syst.* 59 (6), 9010–9025. <http://dx.doi.org/10.1109/TAES.2023.3312626>.
- Song, Y., Wei, L., Yang, Q., Wu, J., Xing, L., Chen, Y., 2023b. RL-GA: A reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem. *Swarm Evol. Comput.* 77, 101236. <http://dx.doi.org/10.1016/j.swevo.2023.101236>.

- Soto, C., Soto, C., Dorronsoro, B., Fraire, H., Cruz-Reyes, L., Gómez-Santillán, C., Rangel, N., 2020. Solving the multi-objective flexible job shop scheduling problem with a novel parallel branch and bound algorithm. *Swarm Evol. Comput.* 53, 100632. <http://dx.doi.org/10.1016/j.swevo.2019.100632>.
- Srinivas, M., Patnaik, L., 1994. Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Trans. Syst. Man Cybern.* 24 (4), 656–667. <http://dx.doi.org/10.1109/21.286385>.
- Sun, K., Zheng, D., Song, H., Cheng, Z., Lang, X., Yuan, W., Wang, J., 2023. Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system. *Expert Syst. Appl.* 215, 119359. <http://dx.doi.org/10.1016/j.eswa.2022.119359>.
- Sundar, S., Suganthan, P.N., Jin, C.T., Xiang, C.T., Soon, C.C., 2017. A hybrid artificial bee colony algorithm for the job-shop scheduling problem with no-wait constraint. *Soft Comput.* 21 (5), 1193–1202. <http://dx.doi.org/10.1007/s00500-015-1852-9>.
- Sutton, R., Barto, A., 1998. Reinforcement learning: An introduction. *IEEE Trans. Neural Netw.* 9 (5), 1054. <http://dx.doi.org/10.1109/TNN.1998.712192>.
- Tang, J., Zhang, G., Lin, B., Zhang, B., 2011. A hybrid algorithm for flexible job-shop scheduling problem. *Procedia Eng.* 15, 3678–3683. <http://dx.doi.org/10.1016/j.proeng.2011.08.689>.
- Teekeng, W., Thammano, A., Unkaw, P., et al., 2016. A new algorithm for flexible job-shop scheduling problem based on particle swarm optimization. *Intell. Manuf.* 21, 18–32.
- Tian, Y., Li, X., Ma, H., Zhang, X., Tan, K.C., Jin, Y., 2023. Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization. *IEEE Trans. Emerg. Top. Comput. Intell.* 7 (4), 1051–1064.
- Tianhua, J., 2018. Flexible job shop scheduling problem with hybrid grey wolf optimization algorithm. *Control Decis.* 33 (3), 503–508. <http://dx.doi.org/10.13195/j.kzyjc.2017.0124>.
- Wang, L., Tang, D.B., 2011. An improved adaptive genetic algorithm based on hormone modulation mechanism for job-shop scheduling problem. *Expert Syst. Appl.* 38 (6), 7243–7250. <http://dx.doi.org/10.1016/j.eswa.2011.11.051>.
- Xing, L.N., Chen, Y.W., Wang, P., Zhao, Q.S., Xiong, J., 2010. A knowledge-based ant colony optimization for flexible job shop scheduling problems. *Appl. Soft Comput.* 10 (3), 888–896. <http://dx.doi.org/10.1016/j.asoc.2009.10.006>.
- Zandieh, M., Madavi, I., Bagheri, A., 2008. Solving the flexible job-shop scheduling problem by a genetic algorithm. *Appl. Sci.* 8, 4650–4655.
- Zhang, G., Gao, L., Shi, Y., 2011. An effective genetic algorithm for the flexible job-shop scheduling problem. *Expert Syst. Appl.* 38 (4), 3563–3573. <http://dx.doi.org/10.1016/j.eswa.2010.08.145>.
- Zhang, Q., Hu, S., 2019. An improved hybrid quantum particle swarm optimization algorithm for FJSP. In: *Proceedings of the 2019 11th International Conference on Machine Learning and Computing. ICMLC '19*, pp. 246–252.